Software Architecture Document (SAD)

Student Task Planner

Name:Yade Başkan

Course:Software Architecture

Instructor:Bahman Arasteh

Date:04.05.2026

Introduction

Purpose

This document describes the architecture of the **Student Task Planner** application.
This document provides a comprehensive architectural overview of the Student Task Planner application. It details the design decisions, structural patterns, and component interactions used to build the system.

Scope

The Student Task Planner is a web-based productivity tool designed for students to manage academic and personal responsibilities. It features secure user authentication, task lifecycle management, and data visualization through integrated charts and calendars.
The Student Task Planner is a web-based application that allows users to:

- Register and login securely
- Create, update, and delete tasks
- Track task status (todo / done)
- Assign deadlines to tasks
- Visualize tasks using charts
- View tasks in a calendar interface
- Filter tasks by status

1.3 Architectural Representation (3-Tier Layered Architecture)

The system follows a Client-Server Architecture:

- Frontend: React (Vite)
- Backend: Node.js (Express)
- Database: SQLite

The system uses REST API communication between frontend and backend.

Architectural Style and Pattern

The Student Task Planner system primarily follows a layered architectural style. The application is separated into three main layers: Presentation Layer, Application Layer, and Data Layer.

The Presentation Layer is implemented using React. It is responsible for the user interface, task list, calendar, dashboard chart, filtering, dark mode, and user interactions.

The Application Layer is implemented using Node.js and Express. It handles REST API requests, authentication, authorization, task operations, and business logic.

The Data Layer is implemented using SQLite. It stores user and task data persistently.

In addition to the layered architectural style, the system also follows the Model-View-Controller (MVC) pattern:

Model: SQLite database, including users and tasks tables.
View: React frontend, including App.jsx and UI components.
Controller: Express backend routes, including authentication and task management endpoints.

The system also uses REST API as the communication style between frontend and backend. HTTP requests such as GET, POST, PUT, and DELETE are used to access and manipulate system resources.

The system does not use SOA or Microservices architecture because the backend is implemented as a single Express server rather than multiple independent services.

Technical Platform:

- React.js (Frontend)
- Node.js + Express (Backend)
- SQLite (Database)
- Chart.js (Visualization)
- React Calendar (Calendar UI)
- JWT (Authentication)

Transactions

The system handles the following operations:

- Task creation
- Task update (status toggle)
- Task deletion

All operations are performed through HTTP requests and stored in the database.

Quality Attributes

Security

- JWT-based authentication

- User-specific data access (user_id)

Performance

- Fast frontend rendering using React
- Lightweight backend using Express

Reliability

- Persistent storage using SQLite

Use-Case View

Main Use Cases

- **User Management:** Secure Registration, Login, and Session handling.
- **Task Management:** Create, Read, Update (Toggle Status), and Delete (CRUD) operations.
- **Visualization:** Interactive Dashboard with Chart.js and a dedicated Calendar UI.

Personalization: Accessibility features including Dark Mode toggle

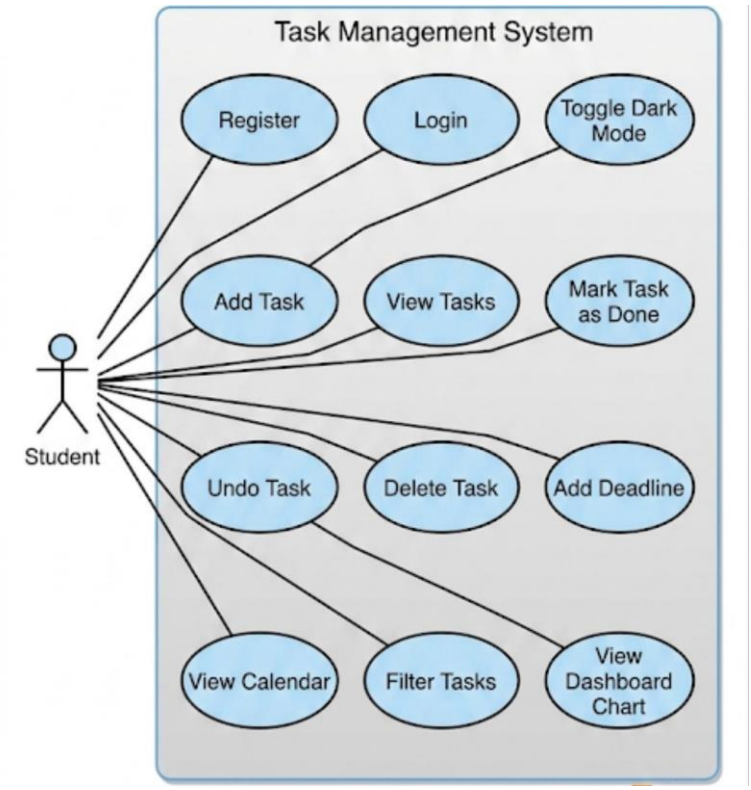
Use-Case Realization Example

Add Task Flow:

- User enters task title and deadline
- Frontend sends POST request
- Backend processes request
- Task is stored in database
- Frontend refreshes task list

Error Flow: Unauthorized Access

- User attempts to access or modify tasks without a valid session.
- The Logic Layer intercepts the request and verifies the **JWT (JSON Web Token)**.
- If the token is missing or invalid, the system returns a **401 Unauthorized** status code.
- The Frontend redirects the user to the login page with a descriptive error message.



Logical View

Overview

The system is divided into three layers:

- Presentation Layer (Frontend)
- Application Layer (Backend)
- Database Layer(Database)

REST API Design

The backend exposes a RESTful interface for seamless communication between the frontend and the database:

POST /register : Creates a new user profile.

POST /login : Authenticates credentials and issues a JWT.

GET /tasks : Fetches tasks filtered by the authenticated user's ID.

POST /tasks : Persists a new task entry.

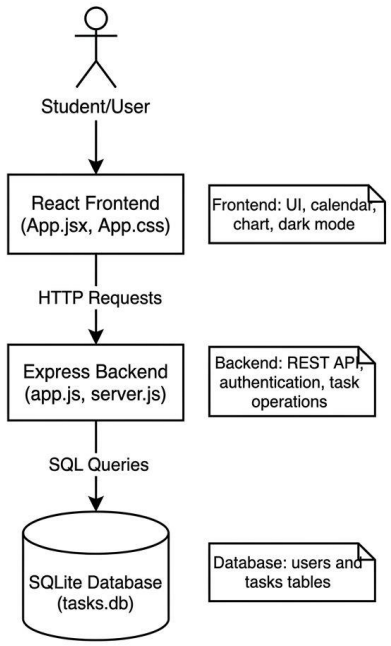
PUT /tasks/:id : Updates existing task metadata (title, deadline, or status).

DELETE /tasks/:id : Permanently removes a task record.

Components

- **Frontend:** App.jsx, UI components (task list, calendar, chart)
- **Backend:** Express server, REST API endpoints
- **Database:** SQLite database storing users and tasks

2. Logical Architecture Diagram



Process View

- The system follows a request-response model:

- User interacts with frontend
- Frontend sends HTTP request
- Backend processes request
- Database is updated/read
- Response returned to frontend

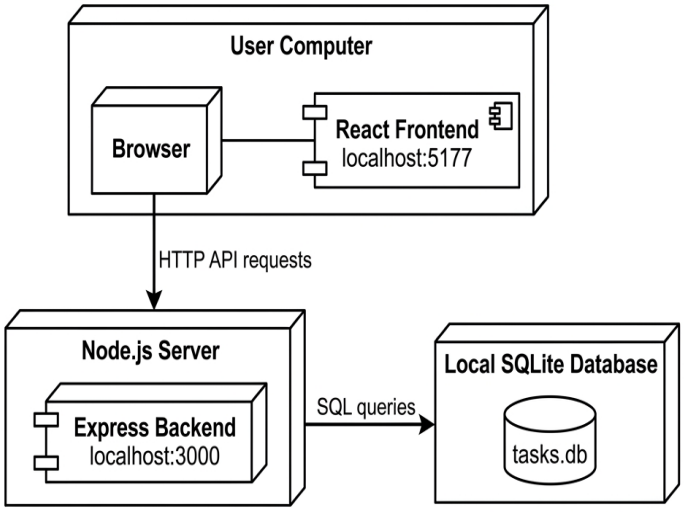
Deployment View

The system runs locally:

- Frontend → localhost:5177
- Backend → localhost:3000
- Database → SQLite (tasks.db)

4. Deployment Diagram

Kutular:



Oklar:

- Browser → Node.js Server: HTTP API requests
- Node.js Server → SQLite Database: SQL queries

Implementation View

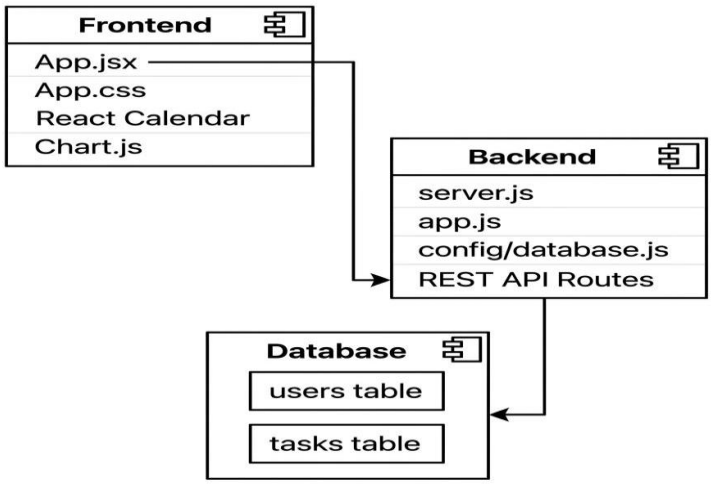
Overview

- The system is implemented using JavaScript technologies:
  - React (Frontend)
  - Node.js (Backend)

Layers

- **Presentation Layer:** Developed with React.js (Vite) for a responsive and dynamic User Interface.
- **Logic Layer:** Powered by Node.js and Express.js to handle business logic and API routing.
- **Data Layer:** Utilizes SQLite for lightweight, persistent, and relational data storage.

5. Component / Implementation Diagram



Data View (Database Design)

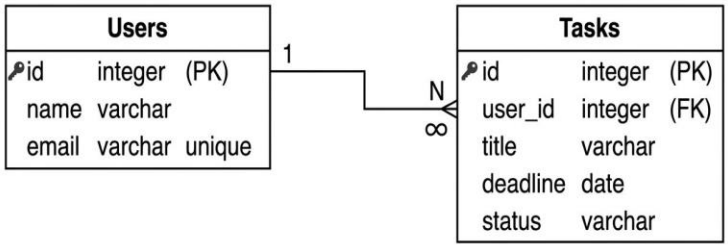
Users Table

- id (Primary Key)
- name
- email
- password

Tasks Table

- id (Primary Key)
- title
- status
- deadline
- user_id (Foreign Key)

Task Management System Database Schema



Security

- JWT token authentication
- User-based data filtering
- Secure password hashing (bcrypt)

Future Improvements

- Mobile application version
- Push notifications for deadlines
- Task sharing between users
- Drag & drop task management
- Cloud deployment (AWS / Vercel)

References

- React Documentation
- Node.js Documentation
- Express.js
- SQLite
- Chart.js